P_OO



(Une image originale représentant le projet)

Hugo chevrette – Cid2A
Vennes
32p

Table des matières

Table des matières

1 INTRODUCTION 3
1.1 OBJECTIFS PRODUIT ET PÉDAGOGIQUE 3
1.2 DESCRIPTION DU DOMAINE ET SOURCES DE DONNÉES 3
2 SPÉCIFICATIONS 3
2.1 TITRE 3
2.2 DESCRIPTION 3
2.3 MATÉRIEL ET LOGICIELS À DISPOSITION 3
2.4 PRÉREQUIS 3
2.5 CAHIER DES CHARGES 3
2.6 LES POINTS SUIVANTS SERONT ÉVALUÉS 3
2.7 VALIDATION ET CONDITIONS DE RÉUSSITE 3
3 PLANIFICATION 4
4 ANALYSE 5
4.1 OPPORTUNITÉS 5
4.2 DOCUMENT D'ANALYSE ET CONCEPTION 5
4.3 BESOINS DE L'UTILISATEUR 6
4.4 MAQUETTES DE L'APPLICATION 6
4.5 DIAGRAMME DE CLASSES 7
4.6 RÔLE DES CLASSES 7
4.7 CONCEPTION DES TESTS 8
4.8 DÉCLARATION D'USAGE DE L'IA 8
5 RÉALISATION 8
5.1 DOSSIER DE RÉALISATION 8
6 TESTS 9
6.1 DOSSIER DES TESTS 9
7 CONCLUSION 9
7.1 BILAN DE LA PLANIFICATION 9
7.2 BILAN DU PRODUIT 9
7.3 BILAN PERSONNEL 10
8 ANNEXES 10
8.1 CAHIER DES CHARGES COMPLET 10
8.1.1 Contraintes 10
8.1.2 Travail à réaliser par l'apprenti 11
8.1.3 Si le temps le permet … 12

1 INTRODUCTION

1.1 Objectifs produit et pédagogique

L'objectif produit est de réaliser une application console en C# permettant de gérer un parking : entrée et sortie des véhicules, affichage de l'état des places, recherche, statistiques et historique des transactions. L'objectif pédagogique est de mettre en pratique les concepts de programmation orientée objet (encapsulation, classes cohérentes, tests unitaires) ainsi que l'utilisation autonome d'un système de versioning (Git/Github).

1.2 Description du domaine et sources de données

Le domaine traité est la gestion d'un parking : un nombre fixe de places, chacune pouvant accueillir un véhicule identifié par sa plaque d'immatriculation. Il n'y a pas de source de données externe : les données (places, véhicules, transactions) sont générées et gérées en mémoire durant l'exécution du programme, aucune base de données n'étant requise pour ce projet.

2 SPÉCIFICATIONS

2.1 Titre

Gestion de parking

2.2 Description

Concevoir et réaliser une simulation de parking

2.3 Matériel et logiciels à disposition

- Un PC ETML
- Accès à Internet

2.4 Prérequis

- Modules de programmation de base
- ICT-320 en cours

2.5 Cahier des charges

Le cahier des charges complet fourni par l'ETML (P_OO – Gestion de parking) est disponible en Annexe, pour ne pas alourdir ce chapitre par une copie du document source.

2.6 Les points suivants seront évalués

- Le rapport
- Les planifications (initiale et détaillée)
- Le journal de travail
- Le code et les commentaires
- Les documentations de mise en œuvre et d'utilisation

2.7 Validation et conditions de réussite

- Compréhension du travail
- Possibilité de transmettre le travail à une personne extérieure pour le terminer, le corriger ou le compléter
- Etat de fonctionnement du produit livré

3 PLANIFICATION

Semaine	1	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**
	1	lecture complete du CdC + questions au CdP si besoin
doc	1	setup Github (repo, README, .gitignore)
	2	squelette des classes Parking, PlaceParking, Vehicule, Ticket (proprietes)
	1	redaction intro - objectifs produit et pedagogique
	2	redaction intro - description du domaine et sources de donnees
	3	analyse fonctionnelle (user stories + maquette console)
	1	planification initiale (ce planning)
	1	relecture + finalisation de la livraison 1 (04.09)
Total semaine	12	**Max. 12**

Semaine	2	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**
code	2	finaliser structure des classes + constructeurs
code	3	entree vehicule - validation format plaque (2 lettres + jusqu'a 6 chiffres)
code	3	entree vehicule - attribution du ticket (heure entree, place, tarif horaire)
doc	2	tests unitaires validation de plaque
	1	commit + push Github
	1	journal de travail
Total semaine	12	**Max. 12**

Semaine	3	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**
code	1	sortie vehicule - saisie plaque ou numero de place
code	3	sortie vehicule - calcul duree (simulee) + montant
doc	2	sortie vehicule - confirmation utilisateur + liberation de la place
	4	affichage etat du parking (plan L=Libre / X=Occupe)
	1	tests unitaires sortie / affichage
	1	journal de travail
Total semaine	12	**Max. 12**

Semaine	4	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**
code	4	recherche d'un vehicule (plaque ou n. place) + affichage du ticket
code	4	statistiques du jour (places occupees, %, montant total, durees)
doc	2	tests unitaires recherche / statistiques
	1	commit + push Github
	1	journal de travail
Total semaine	12	**Max. 12**

Semaine	5	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
code	5	historique des transactions (structure ENTREE / SORTIE)	
code	3	affichage de l'historique formate	
doc	2	tests unitaires historique	
	1	commit + push Github	
	1	journal de travail	
Total semaine	12	**Max. 12**	

Semaine	6	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
code	3	relecture code - conventions techniques ETML (namespace, classes)	
code	3	refactor - eliminer copie/colle, optimiser les structures	
doc	2	completer les tests unitaires manquants	
	2	commentaires de code	
	1	commit + push Github	
	1	journal de travail	
Total semaine	12	**Max. 12**	

Semaine	7	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
code	3	diagramme de classe (UML)	
code	2	redaction de l'explication du diagramme de classe	
	3	rapport de tests (redaction des resultats des tests unitaires)	
	3	chapitre usage de l'IA (min 200 mots) + declaration d'usage	
	1	journal de travail	
Total semaine	12	**Max. 12**	

Semaine	8	
Tâche	**Durée [1/4 h.]**	**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
	2	bilan du deroulement du projet (planning, methodologie, problemes)	
	2	bilan produit (ce qu'on a vs ce qu'on voulait)	
	1	conclusion	
	2	relecture complete du rapport (orthographe, mise en page)	
	1	finaliser le journal de travail	
	2	preparer la release Github (tag, rapport PDF + journal attaches)	
	2	marge / imprevus	
Total semaine	12	**Max. 12**	

4 ANALYSE

4.1 Opportunités

Structurer un programme avec des classes cohérentes (Parking, PlaceParking, Vehicule, Ticket), simuler le temps, faire des tests.

4.2 Document d'analyse et conception

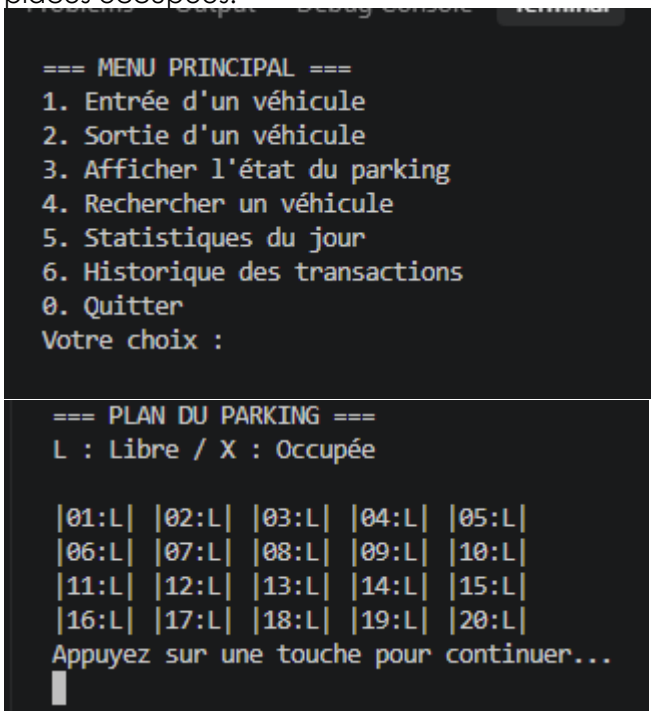
Le programme propose 6 fonctionnalités grâce a un menu console :
Entrée/ sortie/ affichage de l'état du parking/ recherche d'un véhicule/ statistiques du jour / historique des transactions.

4.3 Besoins de l'utilisateur

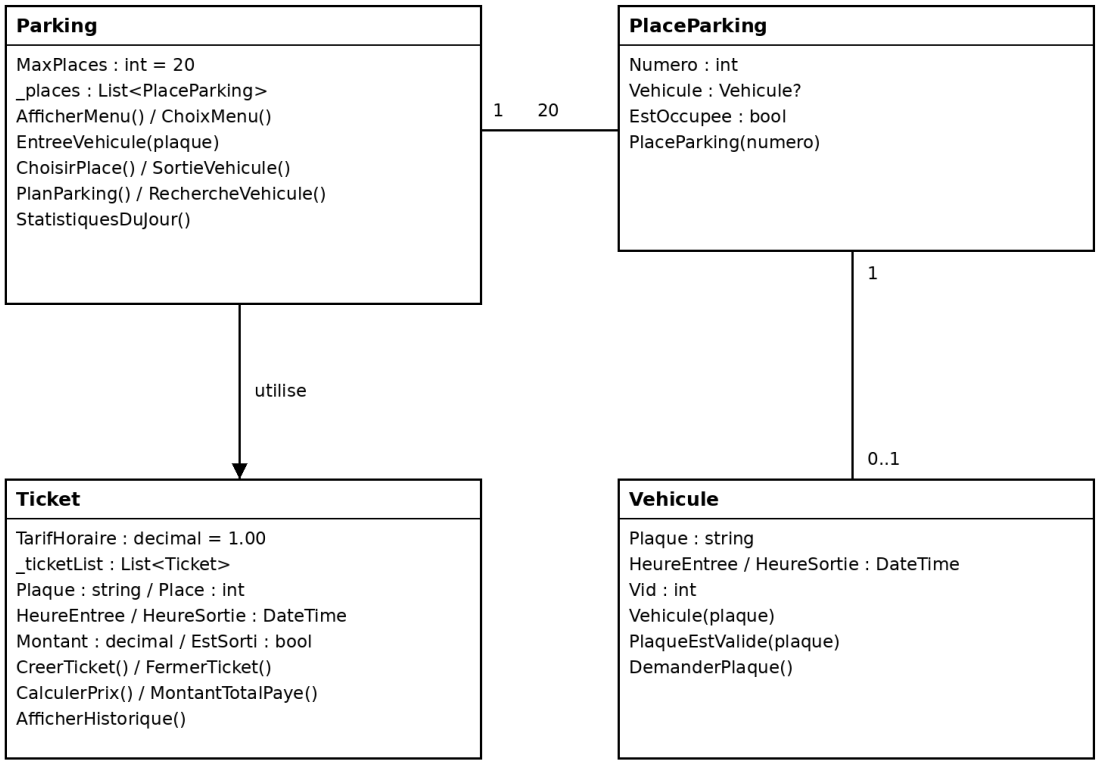
En tant que gestionnaire, je veux enregistrer une plaque pour attribuer une place libre et remettre un ticket.
En tant que gestionnaire, je veux retrouver un véhicule par plaque ou par place pour consulter son stationnement.
En tant que gestionnaire, je veux voir le montant avant de confirmer la sortie pour pouvoir accepter ou annuler.
En tant que gestionnaire, je veux consulter le plan pour repérer les places libres et occupées.
En tant que gestionnaire, je veux consulter le taux d'occupation et les paiements pour suivre l'utilisation du parking.
En tant que gestionnaire, je veux consulter les entrées et les sorties pour retrouver les transactions de la session.

4.4 Maquettes de l'application

Le menu donne accès aux six fonctions. Le plan présente cinq places par ligne. Les écrans ci-dessous sont des maquettes de présentation, avec des exemples de places occupées.



4.5 Diagramme de classes



Parking contient les vingt places. Chaque PlaceParking peut accueillir zéro ou un Vehicule. Le statut occupé dépend de la présence d'un véhicule. Parking utilise Ticket pour enregistrer les passages et calculer les montants. Les tickets terminés restent disponibles pour l'historique et les paiements.
Le tarif est de 1 CHF par heure. Une heure coûte donc 1 CHF et trente minutes coûtent 0,50 CHF. Les secondes sont prises en compte et le montant est arrondi à deux décimales. Le programme conserve ses données uniquement pendant la session et utilise l'heure réelle.

4.6 Rôle des classes

- **Parking (gestion des places et du menu)**

Attribut	**Type**
_places	List<PlaceParking>
MaxPlaces	int (20)

- **PlaceParking** (véhicule, statut libre/occupée)

Attribut	**Type**
Vehicule	Vehicule
EstOccupee	bool
Numero	int

- **Vehicule** (plaque)

Attribut	**Type**
Plaque	string

- **Ticket** (calcul de la durée et du montant).

Attribut	Type
Place	int
HeureEntree	DateTime
HeureSortie	DateTime
TarifHoraire	decimal (1 CHF)

- Le montant total payé est calculé par Ticket, en additionnant les montants des tickets terminés. Parking affiche ensuite ce total dans les statistiques.

4.7 Conception des tests

Les tests automatiques vérifient les plaques, les prix, les places, les recherches, les sorties et les statistiques. Les durées du calcul sont fixées dans les tests pour obtenir des résultats connus. Le détail de la dernière exécution est livré dans Tests/Resultats.txt.

4.8 Déclaration d'usage de l'IA

L'IA (Claude) a été utilisée durant cette phase d'analyse pour structurer la réflexion (planification, idées, explication de concepts POO), en accord avec le niveau d'usage autorisé pour ce projet. Les échanges sont conservés dans la conversation servant de journal de prompts.

5 RÉALISATION

5.1 Dossier de Réalisation

Le projet consiste à développer une application de gestion de parking en C#, utilisable dans une console. Le développement a été réalisé avec Visual Studio sous Windows.
L'application permet de gérer l'entrée et la sortie des véhicules, de leur attribuer une place et de rechercher un véhicule grâce à sa plaque d'immatriculation.
La réalisation a été divisée en plusieurs étapes :

- Création des classes nécessaires à la gestion du parking et des véhicules.
- Attribution d'une place libre lors de l'entrée d'un véhicule.
- Conservation de la plaque, du numéro de place et de l'heure d'entrée.
- Affichage du plan du parking avec **L** pour une place libre et **X** pour une place occupée, à raison de cinq places par ligne.
- Enregistrement de l'heure de sortie et calcul du prix au tarif de 1 CHF par heure, arrondi à deux décimales.
- Ajout de la recherche d'un véhicule par sa plaque.

La classe Ticket regroupe les informations du stationnement : la plaque, la place, l'heure d'entrée et l'heure de sortie. Ce choix permet de conserver ces informations au même endroit.
Pour reprendre le projet, il faut ouvrir le fichier P_OO-Parking.sln dans Visual Studio, puis compiler et lancer l'application.

6 TESTS

6.1 Dossier des tests

Test	Procédure	Résultat attendu
Entrée d'un véhicule	Saisir une plaque et faire entrer le véhicule.	Une place libre est attribuée et l'heure d'entrée est enregistrée.
Plan du parking	Afficher le plan après une entrée.	La place attribuée apparaît avec un **X**, les places libres avec un **L**.
Recherche	Rechercher une plaque présente dans le parking.	Le véhicule et sa place sont retrouvés.
Recherche inconnue	Rechercher une plaque absente.	Un message indique que le véhicule est introuvable.
Sortie d'un véhicule	Faire sortir un véhicule présent.	L'heure de sortie est enregistrée et la place est libérée.
Calcul du prix	Vérifier le montant pour une durée connue.	Le prix correspond au tarif de 1 CHF par heure, arrondi à deux décimales.
Parking complet	Essayer d'ajouter un véhicule lorsque toutes les places sont occupées.	L'entrée est refusée avec un message adapté.

Résultats obtenus : 24 tests réussis, aucun échec. Les cas testés incluent les plaques invalides ou en double, le parking complet, les recherches par plaque et par place, les sorties confirmées ou annulées, les paiements et le menu. Les tests utilisent les classes du programme. La compilation n'a produit ni erreur ni avertissement.

7 CONCLUSION

7.1 Bilan de la planification

La réalisation s'est organisée autour des fonctions principales : l'entrée des véhicules, l'affichage des places et la sortie. La recherche et le calcul du prix ont ensuite complété l'application.

Les semaines 1 à 7 contiennent chacune 12 quarts d'heure saisis. La semaine 8 contient 3 quarts d'heure. Cela représente 21 h 45 sur les 24 heures prévues. Les séances non renseignées et le bilan personnel restent à compléter avec les informations réelles.

7.2 Bilan du produit

Les six fonctions de base et les quatre classes sont présentes. Les méthodes et les variables sont commentées. Les tests automatiques sont livrés avec leurs résultats. La simulation du temps, la sauvegarde après fermeture et les ajouts facultatifs ne sont pas réalisés. Le lancement sur Windows et la release GitHub restent à vérifier.

7.3 Bilan personnel

Ce projet m'a permis de mieux comprendre l'utilisation des classes et des objets en C#. J'ai également travaillé les boucles, les conditions et la gestion des dates pour calculer une durée de stationnement.

Si c'était à refaire, je garderais la séparation des informations dans plusieurs classes. Je prévoirais davantage de tests après chaque fonctionnalité pour repérer les erreurs plus rapidement. Je définirais aussi dès le départ les règles précises du calcul du prix, notamment pour les minutes commencées.

Ce projet m'a appris à construire une application étape par étape et à faire fonctionner plusieurs classes ensemble.

8 ANNEXES

8.1 Cahier des charges complet

Gestion de projet

1. La planification est à faire selon les instructions spécifiques de votre chef de projet.

2. Un journal de travail devra être rendu. L'outil que vous utilisez est libre, mais les caractéristiques suivantes doivent être respectées :

- La structure et la présentation sont claires et soignées.
- Les sources, les fichiers, les répertoires, les commits, et autres sources d'informations concernées par le journal sont référencés.
- L'état et les durées des tâches mentionnées sont précisés.
- Toutes les activités planifiées, les aides extérieures, ainsi que les imprévus et les heures supplémentaires y sont mentionnés.
- Les succès et les échecs sont mentionnés.
- Le travail journalier et son appréciation critique, ainsi que les réflexions y figurent.

8.1.1 Contraintes

1. Le nommage des éléments de code doit être cohérent avec le thème du parking.

2. Le code contient (au moins) une classe pour modéliser chacun des éléments suivants :

- Le parking
- La place de parking
- Un véhicule
- Le ticket

8.1.2 Travail à réaliser par l'apprenti

- Entrée d'un véhicule

Lors de l'entrée d'un véhicule dans le parking, celui-ci doit fournir sa plaque

d'immatriculation.

Vérifier que le numéro de plaque est cohérent : 2 lettres – jusqu'à 6 chiffres.

Attribuer un ticket qui affiche : une heure d'entrée, la place véhicule, le tarif horaire

- Sortie d'un véhicule

Lors de la sortie d'un véhicule l'utilisateur doit saisir sa plaque d'immatriculation ou son

numéro de place.

Le système indique l'heure d'entrée, l'heure de sortie la durée (trouver un moyen de simuler

ce point) et le montant à payer (calculé en fonction de la durée)

Demander à l'utilisateur s'il veut réellement sortir et si oui rendre la place libre.

- Afficher l'état du parking

Afficher un plan du parking avec un moyen d'indiquer quelles sont les places occupées et

quelles sont les places libres.

- Rechercher un véhicule

Demander à l'utilisateur une plaque d'immatriculation ou un numéro de place pour

rechercher le véhicule.

Si trouvé : afficher le ticket (immatriculation, heure d'entrée, durée, prix actuel, n° place)

- Statistiques du jour

Afficher des statistiques liées à l'utilisation actuelle du parking : nombre de place occupées,

% du total, montant total payé, durée de stationnement actuel de tous les véhicules, etc.

- Historiques des transactions

Afficher les transactions effectuées :

- ENTREE (heure d'entrée) – Plaque d'immatriculation (n° place)
- SORTIE (heure de sortie) – Plaque d'immatriculation (n° place, montant payé)

8.1.3 *Si le temps le permet …*

a. Avoir plusieurs types de véhicules dans le parking (avec des tarifs différents)

b. Version Windows Forms

c. Autre proposition personnelle, validée par le CdP